

LECTURE 12 · FINALE

Architecture

Testing · Deployment

The whole picture — twelve weeks on one map.

Dejan Mitov

UACS · CSCI 2450 · Computer Interface Programming · 2026

Twelve weeks ago

You wrote your first `new File("hello.txt")`.

Today you ship a secured REST API.

Today's job

Connect every layer you built in the last five lectures into one mental model.

So when an interviewer asks how it all works — you have the answer ready, in your own code.

No new tools today. Just the ones you already own.

By the end of this hour, you have all of them

Q1

“Walk me through how you'd structure a Spring Boot REST API.”

Q2

“Your booking service must not double-book. How do you make sure?”

Q3

“Hand me your code — how do I run it on a fresh machine?”

Q4

“How do you keep unauthorised users out?”

Each question maps to a lecture you already lived through.

Re-tour the course — then talk about it

1

Tour L8–L11

Quick recap of every layer you built.

2

One stack

How the pieces fit on one diagram.

3

Talk about it

Answer the four interview questions.

4

Project = portfolio

Your repo is your CV.

No new tools, no new acronyms. We make sense of what you already wrote.

The twelve-week timeline

W 1 - 2

Files & I/O

new File() · BufferedReader

W 3

Threads

Runnable · synchronized ·
ExecutorService

W 4

Sockets

Socket · ServerSocket ·
TCP/IP

W 5

JSON

Serialization · Jackson

W 7

Mid-term

Everything so far

W 8

Git · Docker · DB

Version control · containers ·
SQL

W 9

Spring Boot

@SpringBootApplication · IoC
· DI

W 10

REST + JPA

@RestController ·
JpaRepository · errors

W 11

Security

AuthN/AuthZ · JWT · OAuth
concepts

W 12

Today

Putting it all together

PART 1 / 4

Tour the stack

Five lectures. One re-tour. Everything you already own.

Git, Docker, Databases

Git

Version control. Branch, commit, push, pull request.

- `git clone / git push`
- `branch per feature`
- `pull request to merge`

Docker

One image, runs anywhere.

- `Dockerfile = recipe`
- `docker build → image`
- `docker run → container`

PostgreSQL

Relational data. Tables, rows, SQL.

- `CREATE / SELECT / JOIN`
- `primary + foreign keys`
- `transactions`

Interview answer this unlocks

“How would I run your code on a fresh machine? Clone the repo, then ``docker run``. Same image dev → prod.”

Docker — the picture you already drew

1

Dockerfile

The recipe — a text file you write.

2

Image

What `docker build` produces. Immutable, shareable.

3

Container

A running instance. `docker run` makes one.

```
$ git clone https://github.com/you/cip-project.git      # from L8
$ cd cip-project
$ docker build -t cip-project:1.0 .                    # from L8
$ docker run -p 8080:8080 cip-project:1.0              # the API is up
```


Spring Boot — the framework that wires it all

Inversion of Control

"You don't wire your objects. Spring does."

You declare what you NEED. Spring's container looks at your annotations and hands you the right object. You stop writing `new` for components.

*Same idea as plugging an appliance into the wall.
You don't generate the electricity.*

Dependency Injection

"Pass it in. Don't reach out for it."

```
@Service
class FriendService {
    private final FriendRepository repo;

    FriendService(FriendRepository r) {
        this.repo = r;
    }
}
```

Constructor injection. Spring finds the FriendRepository bean and hands it to you. No `new`.

Spring's “magic stickers” — annotations you've used

Annotation	Means	Where you put it in your project
<code>@SpringBootApplication</code>	I'm the main class — start everything	On your XApplication.java (Spring Initializr put it there)
<code>@RestController</code>	I handle HTTP, return JSON	FriendController · JobController · whatever your topic is
<code>@Service</code>	I hold business logic	FriendService — the brain in the middle
<code>@Repository</code>	I talk to the database	Often implicit — JpaRepository carries it
<code>@Entity</code>	Hibernate, make a table for me	On Friend.java — the data class
<code>@Bean</code>	Register this as a component	Inside @Configuration classes when you wire something custom

Six annotations. That's most of Spring Boot in your project right now.

Three layers — the heart of your API

Controller

Talks HTTP

Receives requests, returns JSON.
@RestController, @GetMapping...

Service

Talks logic

Decides what should happen.
Enforces your business invariant.

Repository

Talks database

Saves and finds data.
JpaRepository — Spring writes the SQL.

Interview answer this unlocks

Q1 — “Walk me through how you'd structure a Spring Boot REST API.” Answer: three layers, each does one job. Done.

CRUD ⇔ HTTP — the table you already memorised

Operation	HTTP verb	URL	Success status
Create	POST	/friends	201
List	GET	/friends	200
Find one	GET	/friends/{id}	200
Change	PUT	/friends/{id}	200
Remove	DELETE	/friends/{id}	200/204

When things go wrong — the four numbers you need

- 400 Bad Request — validation failed (your @Valid kicked in)
- 401 Unauthorized — no API key / invalid token (L11 territory)
- 404 Not Found — id doesn't exist
- 409 Conflict — business invariant violated (your project's special rule)

Error handling — one place, one rule

1

Service throws

Your business rule fails →
throw a domain exception.

2

@ControllerAdvice

One class catches them all.
Maps each to a status code.

3

Client gets JSON

Not a stack trace.
A clean error response.

```
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(BookingConflictException.class)
    ResponseEntity<?> handle(BookingConflictException e) {
        return ResponseEntity.status(409).body(Map.of("error", e.getMessage()));
    }
}
```

Security — who's asking, and what can they do?

Authentication

"Who are you?"

You walk in. The doorman checks your ID.

How: password, token, API key, fingerprint.

Authorization

"What can you do?"

You're inside. Which rooms have your name on them?

How: roles, scopes, ownership checks.

AuthN first. You can't decide what they can do until you know who they are.

In your project: a single X-API-Key header, checked by a HandlerInterceptor. Same shape as the real thing.

Tokens, JWT, OAuth — one line each

JWT

I am the messenger. Three Base64 chunks, signed by the server.

OAuth 2.0

I am the rule-maker. I decide which apps can do what on a user's behalf.

OpenID Connect

I am the identity layer. I tell apps WHO the user actually is.

Interview answer this unlocks

Q4 — “How do you keep unauthorised users out?” → *authN at the door, authZ at the room. Tokens, not passwords, on every request.*

PART 2 / 4

One stack

Every piece you've seen, on one diagram.

Open your project — this is what's inside

1	Client	Postman, browser, curl — anything that speaks HTTP
2	Security Interceptor	X-API-Key checked. 401 if missing or wrong.
3	@RestController	Routes the request, validates the body (@Valid).
4	@Service	Business rules. Enforces your invariant.
5	JpaRepository	Save/find. Spring writes the SQL.
6	H2 Database	Rows. In-memory for class, real DB later.

Six layers. One request travels through all of them. Then back.

What happens when POST /friends arrives



When something goes wrong, the same path runs — and breaks early.

- Bad / missing X-API-Key → Interceptor returns 401. Controller never runs.
- Empty firstName → @Valid throws. @RestControllerAdvice returns 400.
- Duplicate email → Service throws DuplicateEmailException → 409.
- Database down → Spring's default exception handler returns 500.

What's actually in there

<code>pom.xml</code>	Maven build. Lists Spring Web, JPA, H2, Validation, springdoc.
<code>src/main/java/.../XApplication.java</code>	@SpringBootApplication. The main(). You never edit this.
<code>.../config/SecurityInterceptor.java</code>	Checks X-API-Key. From L11.
<code>.../web/XController.java + dto/XDTO.java</code>	@RestController + @RequestBody + @Valid. From L10.
<code>.../web/error/GlobalExceptionHandler.java</code>	@RestControllerAdvice. Maps domain exceptions → status codes.
<code>.../service/XService.java</code>	@Service. The brain. Enforces YOUR invariant.
<code>.../domain/X.java</code>	@Entity. The table shape. From L10.
<code>.../repo/XRepository.java</code>	interface ... extends JpaRepository<X, Long>. From L10.
<code>src/main/resources/application.properties</code>	H2 URL, API key, JPA settings.

Nine files. Every one of them mapped to a lecture you sat through.

PART 3 / 4

Talk about it

Four interview questions. Four answers, in your own code.

Walk me through how you'd structure your API

"I use three layers, each with one job."

"Controllers receive HTTP, validate the body, and return JSON. They never call the database."

"Services hold the business rules. They throw domain exceptions when something is wrong."

"Repositories are interfaces extending JpaRepository — Spring writes the implementation."

"Errors go through @RestControllerAdvice so every endpoint speaks the same status codes."

Then open the project. Show the package tree. Done.

You learned this in L10. You wrote it in your project. You can say it in 30 seconds.

How do you keep state correct?

“My project's business invariant is the no-overlap rule.”

“The service checks for an overlapping booking, throws `BookingConflictException` if found.”

“@RestControllerAdvice turns that into a 409. I have a unit test that proves it.”

Find YOUR invariant category — say it the same way every time

Concurrency

Parking, Inventory, Auction — @Transactional + lock the row.

Time interval

Restaurant, Hotel, Doctor — NOT ($\text{newEnd} \leq \text{oldStart}$ OR $\text{newStart} \geq \text{oldEnd}$).

State machine

Order, Ticket workflow — `map<from, allowedNext>`.

How do I run your code on a fresh machine?

“Clone, build, run. Three commands.”

“The repo has a Dockerfile from L8. Anyone with Docker installed gets the same running app.”

“The H2 database is in-memory, so no separate DB to install. Production would be one config change.”

```
$ git clone https://github.com/you/cip-project.git
$ cd cip-project && docker build -t cip-project:1.0 .
$ docker run -p 8080:8080 cip-project:1.0

# Now open http://localhost:8080/swagger-ui.html — the API is up.
```

You shipped it in L8. You wrote three commands. You can say them in your sleep.

How do you keep unauthorised users out?

“Two layers: authentication first, authorization second.”

“For class my project uses an X-API-Key header, checked by a HandlerInterceptor — same place in the request flow as a real auth filter.”

“In production I'd swap it for JWT — the client logs in once, gets a signed token, sends it on every request as Authorization: Bearer <jwt>.”

“For authorization I'd add roles — @PreAuthorize on the methods that only admins should call. Same Spring, harder mechanism.”

Mention the upgrade path. Shows you understand authentication is layered, not a checkbox.

Bonus points: passwords are hashed with BCrypt. Never store plain text. That's an L11 takeaway you carry forever.

PART 4 / 4

Project = portfolio

Your repo is your resume.

Your project rubric is an interview checklist

10+ REST endpoints

"Design a CRUD API"

Soft delete + audit

"How do you avoid losing data?"

@RestControllerAdvice

"How do you handle errors centrally?"

Business invariant

"How do you keep state correct?"

Pagination on every list

"What happens at 1M rows?"

Swagger + README

"How does your team use the API?"

API key + interceptor

"How do you authenticate?"

Finish the rubric → you've already answered eight interview questions.

Five sections, in order

What it does

One paragraph in plain English. The domain. The problem your API solves.

How to run it

git clone + docker run. One screenshot of Swagger up at /swagger-ui.html.

Authentication

X-API-Key: demo-key-2026 on every request. Missing or bad → 401.

Five example calls

Curl one of each — create, list, find, change, your invariant violation returning 409.

The invariant

One sentence: "The system never allows X." One paragraph: "I enforce it with [@Transactional/state map/predicate]."

4 things any reviewer will catch

⚠ Hard-coded API key in code

Read it from `application.properties`.

⚠ Soft delete forgotten on read

Aggregations include deleted rows. Add `deleted_at IS NULL`.

⚠ Missing pagination

`GET /things` returns everything. Use `Pageable`.

⚠ Invariant tested only happy-path

Show the test that proves the rule REJECTS.

What changed in your hands

Week 1 — what you wrote

```
FileWriter w = new FileWriter("hello.txt");  
w.write("Hello world"); w.close();
```

Today — what you ship

```
@RestController  
@RequestMapping("/api/v1/friends")  
class FriendController(FriendService svc) {  
    @PostMapping  
    Friend create(@RequestBody @Valid FriendDTO d) { ... }  
} // three layers · invariant · validated · containerised
```

Six things to carry out of this room

1 Three layers

Controller / Service / Repository. Each with one job. You can draw it.

3 REST is verbs + URLs

GET POST PUT DELETE — 200 201 400 404 409. Pick the right one.

5 Auth is a layer

X-API-Key today. JWT tomorrow. Same place in the request flow.

2 Spring wires it

@SpringBootApplication starts. Annotations declare. Spring connects.

4 Errors are centralised

@RestControllerAdvice. One place. Every controller. Consistent JSON.

6 Docker ships it

One image. Same code. Laptop, classroom, production.

Within the same Spring Boot world

1

Real database

Swap H2 for PostgreSQL. Run your project against it. Same JPA code.

2

Flyway migrations

Versioned schema changes. Production tables don't get re-created.

3

Login + JWT

Replace X-API-Key with a real login flow. Same place in the request.

4

Tests with @WebMvcTest

Spring's testing slices. Prove your endpoints return the right status.

T H A N K Y O U

For the questions,

the bugs that taught more than the lectures.

Build the project well. Push it to GitHub. Send me the link

Dejan Mitov · Ivana Rancic · UACS · CSCI 2450 · 2026

ONE LAST TIME

Questions?

WHAT'S NEXT

Final exam — after Reading Week

Multiple choice + true/false + practical programming. Closed book.

FINAL PROJECT

One topic. ≥ 10 endpoints. Invariant.

Live demo before the final. Make it the project you put on your CV.